Architecture Document – Resilient SRT Streamer

1)System Overview

The system is a **CLI-based video streaming pipeline** built using GStreamer in C++. It ingests a local media file, processes it into a low-latency H.264 stream, and transmits it over the **SRT protocol** while maintaining **resilience and observability**.

Key Objectives

- Reliable real-time streaming over SRT
- Continuous output even on source failure
- Real-time network visibility (RTT, packet loss)
- Configurable encoding parameters

High-Level Pipeline

```
File Source → Decode → Queue ┐
                             ├→ Input Selector → Queue → Encoder → Mux → Queue → SRT Sink
Fallback Source → Queue ┘
```

Core Components

Component	Role
`filesrc`	Reads input file
`decodebin`	Decodes input dynamically
`videotestsrc`	Generates fallback stream
`input-selector`	Switches between main and fallback
`x264enc`	Encodes to H.264

Component	Role
`mpegtsmux`	Packs into MPEG-TS
`srtsink`	Streams via SRT
`queue`	Buffers to smooth data flow

2)Data Model

The system processes **media buffers (frames)** flowing through the pipeline.

Data Representation

- **Input:** Encoded container (MP4/MKV)
- **Intermediate:** Raw video frames (`video/x-raw`)
- **Encoded Output:** H.264 bitstream
- **Transport Format:** MPEG-TS packets
- **Network Layer:** SRT packets over UDP

3)Execution Model

The system follows an **event-driven, asynchronous execution model**.

Pipeline Execution

- GStreamer pipeline runs in its own internal threads
- Each element processes data independently
- Queues decouple stages for parallelism

Event Flow

1. File is read and decoded
2. Frames flow through selector
3. Encoder processes frames

4. Output is muxed and sent via SRT
5. Stats loop periodically queries SRT metrics
6. Bus listens for EOS or ERROR events

4) Validation Strategy

Validation was performed at multiple levels:

Functional Validation

- Verified successful streaming using:

```
gst-launch-1.0 srtsrc ...
```

- Confirmed:
  - video playback
  - correct decoding

Observability Validation

- Verified real-time stats:
  - RTT updates
  - Packet loss values

Resilience Testing

Test scenarios:

- Input file ends → fallback activated
- Corrupted input → fallback activated
- Verified:
  - no stream interruption
  - SRT connection remains active

Configuration Validation

- Tested CLI parameters:
  - bitrate
  - preset
  - GOP
- Ensured:
  - correct propagation to encoder
  - stable output

5)Tradeoffs

Latency vs Quality

- Used:
  - `zerolatency` tuning
  - `ultrafast` preset

Tradeoff:

- Lower latency
- Reduced compression efficiency

Buffering vs Responsiveness

- Introduced queues for stability

Tradeoff:

- Slight buffering delay
- Improved robustness

CPU vs Compression

- Ultrafast preset reduces CPU load

Tradeoff:

- Higher bitrate needed for same quality

6)If Given More Time

Adaptive Bitrate (ABR)

- Dynamically adjust bitrate based on:
  - packet loss
  - RTT

Receiver-Side Metrics

- Build receiver CLI
- Compare sender vs receiver stats

Smarter Fallback

- Replace test pattern with:
  - static image
  - custom “No Signal” frame

Advanced CLI Parsing

- Use library like `cxxopts`
- Add validation and help menus

Logging & Monitoring

- Structured logs (JSON)
- Integration with monitoring systems

Multi-Bitrate Streaming

- Simultaneous outputs (ABR ladder)

Fault Injection Testing

- Simulate:
  - packet loss
  - jitter
  - latency spikes